

UNIVERSITY OF NOTTINGHAM

School of Computer Sciences

Computer Security (COMP3006)

Submitted: May 2024

Submitted by:

Nderon Beqiri (20365426)

Word Count:

1200

Basics- Passwords & Software

To begin with, the simplest step I can take as the administrator is to ensure my password itself is unique, and incredibly strong, prone to brute-force, dictionary and other attacks. This is to prevent hackers from getting access to the administrative account and being able to use "sudo", change the root password and get access to the root account, as we did with Glenn in Lab 6. To take further measures, we can additionally disable direct root login. This adds an extra layer of security by preventing attackers from directly targeting the root account.

I would further enforce rules for strong passwords as we learnt in Lab 3 such as a minimum length, uppercase letters, numbers and special characters to fend off users being hacked through their simple passwords e.g. Simon- simon55. There will be a failed login attempt limit and users should also have to change their password every 60 days, receiving warnings of the failed login attempt limit being hit and warnings that a password should be changed 7 days prior to expiration. After 7 days with an expired password, an account is locked.

The passwords within the config files need to be hashed as through the phpMyAdmin config file we were able to find the credentials that log us into the phpMyAdmin root account and retrieve Glenn's password and crack it via "Hashcat" (10.0.2.30 web browser) and the "rockyou.txt" file from Lab 3. This ensures that EVEN IF (worst case) an attacker gains access to these files, the passwords remain protected from unauthorised retrieval.

Furthermore, as discussed in the lecture Authentication, installing software updates is the most important safety practise, therefore I would ensure that all system packages and software are up-to-date, patching security vulnerabilities (if given more time). Running the code *'apt list –upgradable'*, we see a variety of **"trusty-security"** updates available on the lab Ubuntu server which we can install using *'sudo apt upgrade'*, as newer updates are less likely to have known exploits targeting them.

Through Lab 6 we were able to see that "the server is running ProFTPD 1.3.5, which is vulnerable to a remote code execution vulnerability". Updating this to a later version ensures that any known vulnerabilities e.g. the bug in the "mod_copy module" are patched.

IPTABLES & Firewalls

To check the current iptables rules we use *'sudo iptables -L'* we got the below output.

```
glenn@ubuntu:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source               destination

Chain FORWARD (policy ACCEPT)
target     prot opt source               destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source               destination
```

Firstly, it's advisable to revise the default policies for the INPUT, FORWARD, and OUTPUT chains from ACCEPT to DROP as our permissive policy has no rules preventing dangerous services. This change ensures that unless explicitly allowed, all inbound, outbound, and forwarded traffic is rejected, reducing the attack surface. Specific rules should be crafted to permit only essential traffic, listing certain ports to allow access, while denying all other traffic by default (restrictive policy). For example, a packet must have a destination port that aligns with SSH e.g. port 22. Another rule can be we allow incoming connections from established communications as learned in Lab 4 , since these

connections have already been verified as legitimate through the initial handshake process, and as a result they are less likely to be malicious or unauthorised.

Looking at the server, we can see the status of the firewall via *'sudo ufw status'*. This returns **"Status: inactive"**. To fix this, we can activate the Uncomplicated Firewall (UFW) by running the command *'sudo ufw enable'*. This will activate UFW and start applying the firewall rules defined in its configuration. After enabling UFW, I restarted the system and used *'sudo ufw status'* again to verify that the firewall is now active and protecting the system. It rightfully returned **"Status: active".**

We can now enter the command *'sudo ufw status verbose'* to see the firewall rules that are set.

```
glenn@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip
```

Linking to the Port Management section, we would like to enforce further rules on the firewall as we did in Lab 4. The firewall is on currently **"deny (incoming)"**, meaning any attempt to connect to services running on the system from external sources will be denied unless explicitly allowed. Some examples of what to allow include SSH and HTTPS to restrict incoming and outgoing traffic to only necessary ports and services to prevent attackers from gaining access to the server. This aids in least privilege means allowing only the specific ports and services that are required for the server's intended functionality.

```
glenn@ubuntu:~$ sudo ufw allow ssh
Rule added
Rule added (v6)
glenn@ubuntu:~$ sudo ufw allow https
Rule added
Rule added (v6)
```

```
glenn@ubuntu:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22                         ALLOW IN    Anywhere
443                        ALLOW IN    Anywhere
22 (v6)                    ALLOW IN    Anywhere (v6)
443 (v6)                   ALLOW IN    Anywhere (v6)
```

Furthermore, we can take inspiration from MySQL and reject any connection not from the same machine, whitelisting IP addresses that are allowed to access specific services and block all other incoming connections, preventing port scanning attempts.

Port Management

Through Lab 6, we see that the exploit functions effectively on Telnet versions.

```
glenn@ubuntu:~$ dpkg -l | grep telnetd
ii  telnetd                          0.17-36build2                          amd64
      The telnet server
```

As confirmed by the command *'dpkg -l | grep telnetd'* on the Ubuntu server, the above screenshot indicates Telnet usage. We can improve security by transitioning from Telnet to SSH. By using SSH over Telnet, we mitigate the risks associated with unauthorised access and potential data breaches due to its encryption, safeguarding sensitive information.

Looking at the ports in use via *'sudo netstat -tuln'* we get the output below.

```
glenn@ubuntu:~$ sudo netstat -tuln
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
tcp        0      0 0.0.0.0:3306            0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:139             0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:21              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:23              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:25              0.0.0.0:*               LISTEN
tcp        0      0 0.0.0.0:445             0.0.0.0:*               LISTEN
tcp6       0      0 :::139                  :::*                    LISTEN
tcp6       0      0 :::80                   :::*                    LISTEN
tcp6       0      0 :::22                   :::*                    LISTEN
tcp6       0      0 :::25                   :::*                    LISTEN
tcp6       0      0 :::445                  :::*                    LISTEN
udp        0      0 10.0.2.255:137          0.0.0.0:*
udp        0      0 10.0.2.30:137           0.0.0.0:*
udp        0      0 0.0.0.0:137             0.0.0.0:*
udp        0      0 10.0.2.255:138          0.0.0.0:*
udp        0      0 10.0.2.30:138           0.0.0.0:*
udp        0      0 0.0.0.0:138             0.0.0.0:*
```

Open ports create potential entry points for malicious actors to exploit vulnerabilities in the system, so we need to either close them or change them for improved security.

Looking at some of these ports, Port 21 (FTP) and port 23 (Telnet) pose significant security risks due to transmitting data in plaintext, so I would close them and rely on more secure alternatives such as Port 22 (SFTP & SSH). Further research revealed an outdated version of OpenSSH, so again updates are necessary. Port 80 (HTTP) would be redirected to port 443 for HTTPS traffic to encrypt data in transit and protect against potential security vulnerabilities associated with unencrypted HTTP traffic e.g. protecting against eavesdropping and man-in-the-middle attacks.

Intrusion Detection System

We can search if we have any active IDS within the server to detect possible intrusion attempts. As seen below we do not.

```
glenn@ubuntu:~$ ps aux | grep ids
glenn       1958  0.0  0.1  11760  2240 tty1     S+   12:16   0:00 grep --color=auto ids
```

We can install an IDS discussed in the lecture Intrusion Detection such as Snort via *"sudo apt-get install snort",* which contains network intrusion detection capabilities, enhancing the server's security by promptly identifying and alerting us as the administrator to potential threats. It's important to configure and monitor the IDS effectively to distinguish between genuine threats and false positives.

Conclusion

In conclusion, as the admin, it is not only important to incorporate these steps but also to maintain, monitor and update the security standard. For example, we need to do regular reviews and adjustment of firewall rules to adapt to evolving threats. Additionally, we recognise the significance of continuously fine-tuning IDS configurations to minimise both false positives and false negatives, thus enhancing our server's overall security.

Upon applying all of these changes, we can proceed with conducting comprehensive penetration testing through methods such as in Lab 6 where we used “meterpreter”, to validate the effectiveness of our security, thus enabling us to identify areas for improvement. With the enforcement of some of the above rules, we were not able to run the exploit successfully on the server, proving our security measures to be a great success.